

Ex No : 01 A**PROGRAM TO DEMONSTRATE OPENMP FORK-JOIN****DATE :****PARALLISIM****AIM:**

To demonstrate the OpenMP Fork-Join model with work sharing using a simple C program.

Definition of OpenMP:

OpenMP (Open Multi-Processing) is an application programming interface (API) that supports multi-platform shared-memory parallel programming in C, C++, and Fortran. It is designed to allow developers to parallelize their code with minimal changes, making it easier to take advantage of multi-core processors and shared-memory architectures.

OpenMP provides a set of compiler directives, library routines, and environment variables that influence run-time behavior. It is particularly well-suited for parallelizing loops and sections of code that can be executed concurrently.

OpenMP Fork-Join Model and OpenMP Constructs:**OpenMP Fork-Join Model**

OpenMP (Open Multi-Processing) is a parallel programming API that enables multi-threading in C, C++, and Fortran programs. It follows a **Fork-Join model**, which consists of two main phases:

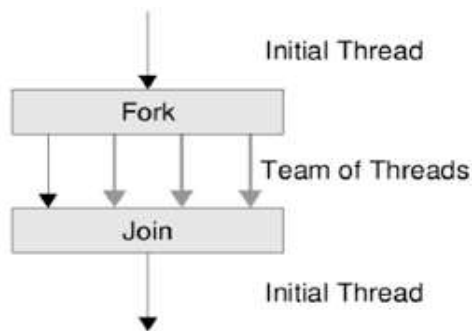
Fork Phase:

- The program starts execution with a single thread (master thread).
- When a parallel region is encountered, the master thread **forks** into multiple threads.
- These threads execute the parallel region concurrently.

Join Phase:

- After completing the parallel region, all spawned threads **join** back into the master thread.
- Execution continues sequentially after the parallel region.
- This model allows programs to exploit parallelism while ensuring synchronization after parallel execution.

Architecture of OpenMP Fork-Join Model:



OpenMP Constructs:

OpenMP provides several constructs to control parallel execution. Some key constructs include:

1. #pragma omp parallel (Parallel Region)

This directive creates a parallel region where multiple threads execute the code block. Each thread gets an identical copy of the code to execute.

Example:

```
#include <stdio.h>
#include <omp.h>
int main() {
    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        printf("Hello from thread %d\n", id);
    }
    return 0;
}
```

OUTPUT:

```
PS C:\Users\arivu\OneDrive\Documents\MAP Lab> ./1a1.exe
Hello from thread 2
Hello from thread 0
Hello from thread 1
Hello from thread 3
```

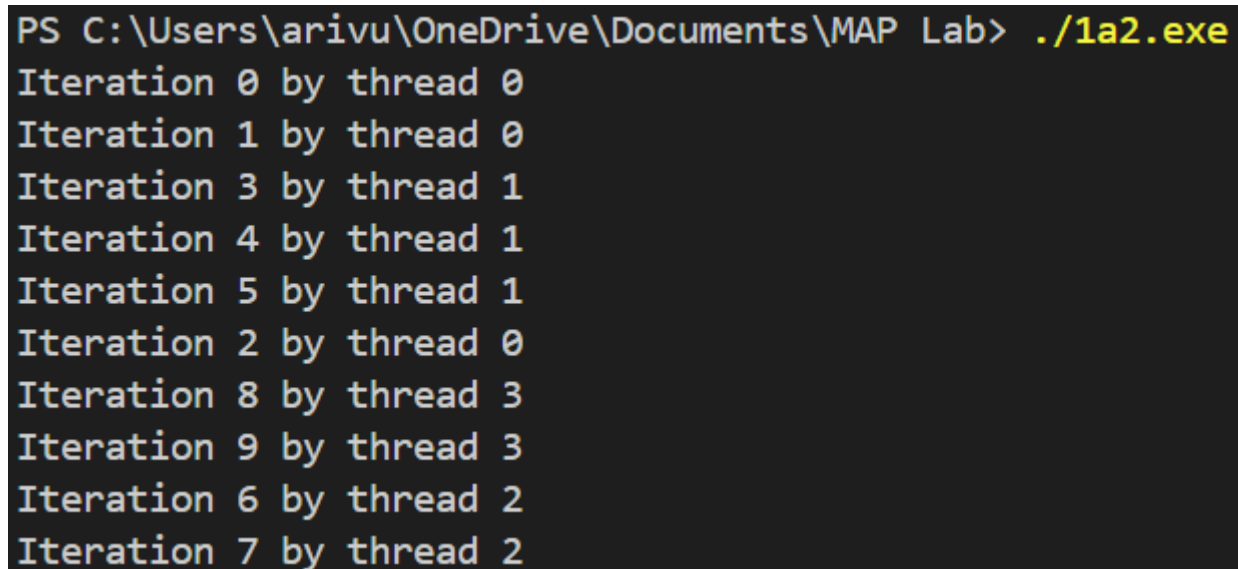
2. #pragma omp for (Parallel Loop)

Used to distribute iterations of a loop across multiple threads. Should be inside a #pragma omp parallel block.

Example:

```
#include <stdio.h>
#include <omp.h>
int main() {
    int i;
    #pragma omp parallel for
    for (i = 0; i < 10; i++) {
        printf("Iteration %d by thread %d\n", i, omp_get_thread_num());
    }
    return 0;
}
```

OUTPUT:



```
PS C:\Users\arivu\OneDrive\Documents\MAP Lab> ./1a2.exe
Iteration 0 by thread 0
Iteration 1 by thread 0
Iteration 3 by thread 1
Iteration 4 by thread 1
Iteration 5 by thread 1
Iteration 2 by thread 0
Iteration 8 by thread 3
Iteration 9 by thread 3
Iteration 6 by thread 2
Iteration 7 by thread 2
```

3. #pragma omp critical (Critical Section)

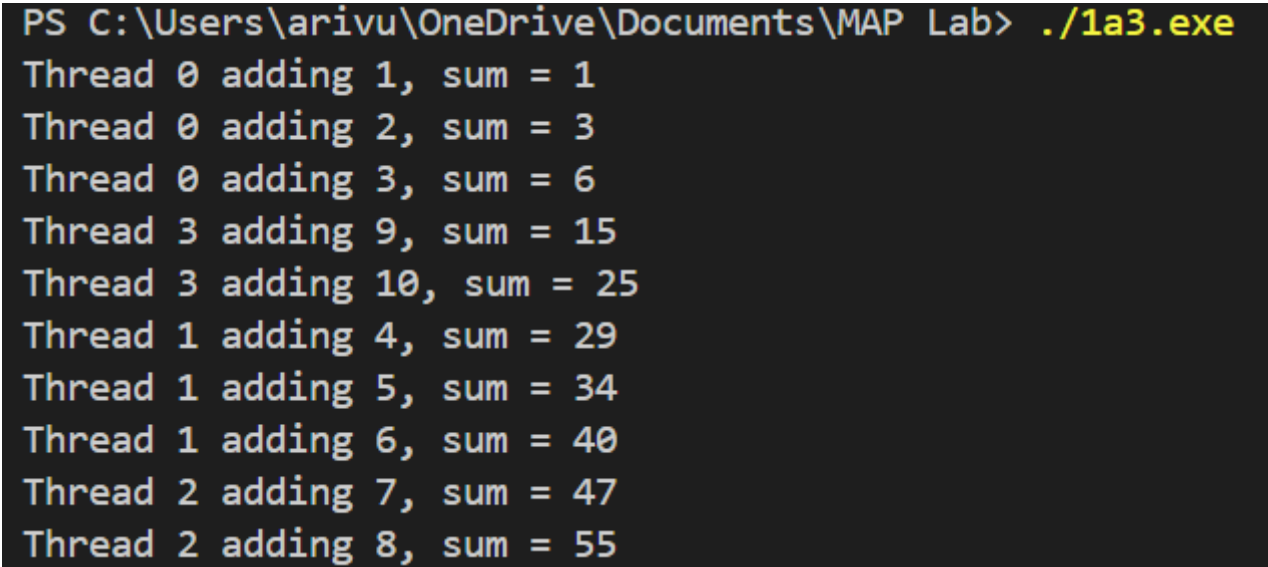
Ensures only one thread executes a particular block of code at a time. Used to prevent race conditions when updating shared resources.

Example:

```
#include <stdio.h>
```

```
#include <omp.h>

int main() {
    int sum = 0;
    #pragma omp parallel for
    for (int i = 1; i <= 10; i++) {
        #pragma omp critical
        {
            sum += i;
            printf("Thread %d adding %d, sum = %d\n", omp_get_thread_num(), i, sum);
        }
    }
    return 0;
}
```

OUTPUT:

```
PS C:\Users\arivu\OneDrive\Documents\MAP Lab> ./1a3.exe
Thread 0 adding 1, sum = 1
Thread 0 adding 2, sum = 3
Thread 0 adding 3, sum = 6
Thread 3 adding 9, sum = 15
Thread 3 adding 10, sum = 25
Thread 1 adding 4, sum = 29
Thread 1 adding 5, sum = 34
Thread 1 adding 6, sum = 40
Thread 2 adding 7, sum = 47
Thread 2 adding 8, sum = 55
```

RESULT:

Thus the OpenMP Fork-Join model with work sharing using a simple C program.

Ex No : 01 B**DATE :****OPENMP FORK-JOIN WITH WORK SHARING****AIM:**

To write a simple Program to demonstrate an OpenMP Fork-Join with work sharing.

ALGORITHM:

Step 1: Start the Program

Step 2: Initialize n = 100

Step 3: Initialize sum = 0

Step 4: Create an array arr[n]

Step 5: For i from 0 to n-1 do: arr[i] = i + 1

Step 6: Start parallel region: Initialize thread_sum = 0

Step 7: For i from 0 to n-1 in parallel do: thread_sum += arr[i]

Step 8: End parallel region

Step 9: Use atomic operation to update sum: sum += thread_sum

Step 10: Print "Total Sum = ", sum

Step 11: End the Program.

PROGRAM:

```
#include <omp.h>
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int n = 100;
```

```
    int sum = 0;
```

```
    int arr[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        arr[i] = i + 1;
```

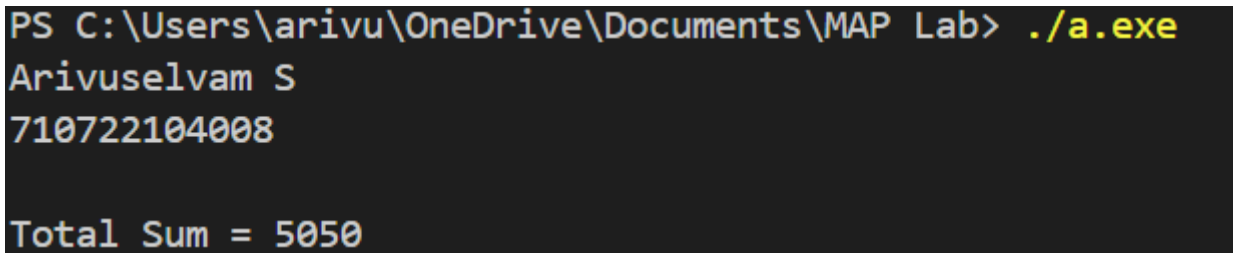
```
    }
```

```
    #pragma omp parallel
```

```
    {
```

```
        int thread_sum = 0;
```

```
#pragma omp for
for (int i = 0; i < n; i++) {
    thread_sum += arr[i];
}
#pragma omp atomic
sum += thread_sum;
}
printf("Total Sum = %d\n", sum);
return 0;
}
```

OUTPUT:

```
PS C:\Users\arivu\OneDrive\Documents\MAP Lab> ./a.exe
Arivuselvam S
710722104008

Total Sum = 5050
```

RESULT:

Thus simple Program for OpenMP Fork-Join with work sharing was executed successfully.

Ex No : 01 C**DATE :****PERFORMANCE TUNING AND OPTIMIZE OPENMP****AIM:**

To write a simple Program to demonstrate the Performance Tuning and Optimize OpenMP.

ALGORITHM:

Step 1: Start the Program

Step 2: Initialize variables

SET n = 1000000

SET sum = 0

DECLARE array arr[n]

Step 3: Initialize the array :arr[i] = 1

Step 4: Start timing

SET start_time = omp_get_wtime()

Step 5: Parallel summation

#pragma omp parallel

BEGIN

Declare thread-local variable for partial sum

SET thread_sum = 0

Distribute work among threads

#pragma omp for reduction(+:sum) schedule(dynamic, 1000)

FOR i FROM 0 TO n - 1 DO

thread_sum += arr[i] // Each thread calculates its partial sum

END FOR

Combine results (handled by reduction clause)

END

Step 6: End timing

SET end_time = omp_get_wtime()

Step 7: Output the result

PRINT "Sum = ", sum

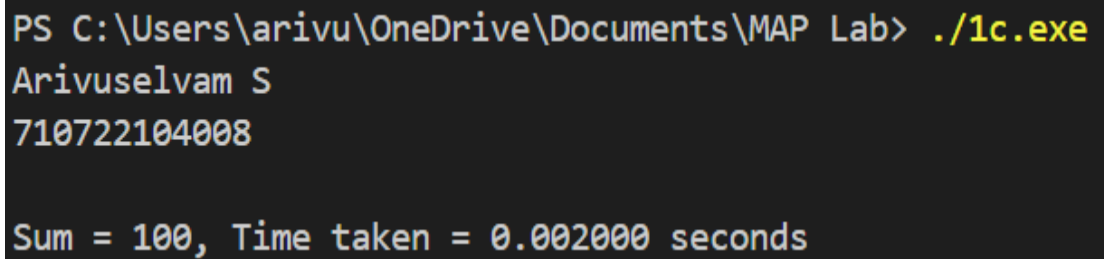
```
PRINT "Time taken = ", end_time - start_time, " seconds"
```

Step 8: End the Program

PROGRAM:

```
#include <omp.h>
#include <stdio.h>
int main() {
    int n = 1000000, sum = 0;
    int arr[n];
    for (int i = 0; i < n; i++) arr[i] = 1; // Initialize array
    double start_time = omp_get_wtime();
    #pragma omp parallel for reduction(+:sum) schedule(dynamic, 1000)
    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }
    double end_time = omp_get_wtime();
    printf("Sum = %d, Time taken = %f seconds\n", sum, end_time - start_time);
    return 0;
}
```

OUTPUT:



```
PS C:\Users\arivu\OneDrive\Documents\MAP Lab> ./1c.exe
Arivuselvam S
710722104008

Sum = 100, Time taken = 0.002000 seconds
```

Result:

Thus the Program to demonstrate the Performance Tuning and Optimize OpenMP was executed successfully.

Ex No : 02 A	SEQUENTIAL IMPLEMENTATION OF MATRIX-VECTOR MULTIPLICATION
DATE :	

AIM:

To Create a program that computes a simple matrix-vector multiplication $b=Ax$, either in C/C++.

ALGORITHM:

Step 1: A: The matrix of size $M \times N$

Step 2: x: The vector of size N

Step 3: b: The result vector of size M

Step 4: Check if the number of columns in matrix A (N) matches the size of vector x (N). If they are not equal, return an error, as matrix-vector multiplication is not possible.

Step 5: Initialize the elements of vector b to zero.

Step 6: For each row i in the matrix A ($0 \leq i < M$): a. Initialize the element $b[i]$ to zero. b.

Step 7: For each column j in the matrix A ($0 \leq j < N$): i. Add the product of $A[i][j]$ and $x[j]$ to $b[i]$.

Step 8: The resulting vector b contains the result of the matrix-vector multiplication ($b = Ax$).

PROGRAM:

```
#include <stdio.h>
```

```
#define N 4 // Matrix Size
```

```
void sequential_matvec(double matrix[N][N], double vector[N], double result[N]) {
```

```
    for (int i = 0; i < N; i++) {
```

```
        result[i] = 0;
```

```
        for (int j = 0; j < N; j++) {
```

```
            result[i] += matrix[i][j] * vector[j];
```

```
        }
```

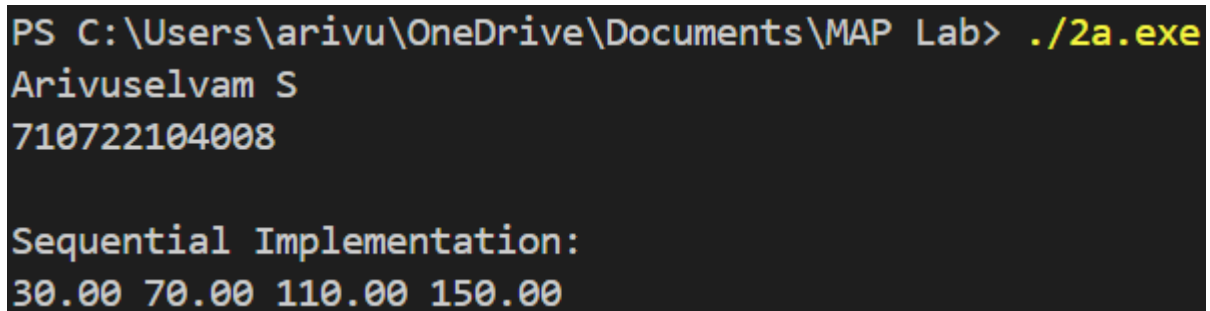
```
    }
```

```
}
```

```
void print_vector(double result[N]) {
```

```
        for (int i = 0; i < N; i++) {
            printf("%.2f ", result[i]);
        }
        printf("\n");
    }
int main() {
    double matrix[N][N] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12},
        {13, 14, 15, 16}
    };
    double vector[N] = {1, 2, 3, 4};
    double result[N];
    printf("Sequential Implementation:\n");
    sequential_matvec(matrix, vector, result);
    print_vector(result);
    return 0;
}
```

OUTPUT:



```
PS C:\Users\arivu\OneDrive\Documents\MAP Lab> ./2a.exe
Arivuselvam S
710722104008

Sequential Implementation:
30.00 70.00 110.00 150.00
```

RESULT:

Thus, creation of simple matrix-vector multiplication $b=Ax$, either in C/C++ was created and output is verified.

Ex No : 02 B	PARALLELIZING IMPLEMENTATION OF MATRIX - VECTOR MULTIPLICATION USING OPENMP
DATE :	

AIM:

To create a program that computes a parallelizing implementation of matrix-vector multiplication $b=Ax$, using OpenMP.

ALGORITHM:

Step 1: Initialize matrix A and vector V.

Step 2: Use OpenMP to run different rows in parallel: Assign each row computation to a different thread.

Step 3: Store the results in R.

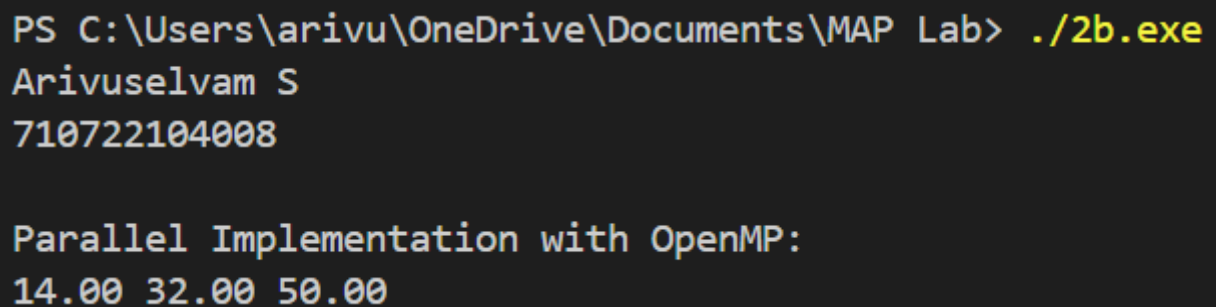
Step 4: Print the result.

PROGRAM:

```
#include <stdio.h>
#include <omp.h>
#define N 3 // Matrix Size
void parallel_matvec(double matrix[N][N], double vector[N], double result[N]) {
    #pragma omp parallel for // OpenMP parallelization
    for (int i = 0; i < N; i++) {
        result[i] = 0;
        for (int j = 0; j < N; j++) {
            result[i] += matrix[i][j] * vector[j];
        }
    }
}
void print_vector(double result[N]) {
    for (int i = 0; i < N; i++) {
        printf("%.2f ", result[i]);
    }
}
```

```
    printf("\n");
}
int main() {
    double matrix[N][N] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };
    double vector[N] = {1, 2, 3};
    double result[N];
    printf("Parallel Implementation with OpenMP:\n");
    parallel_matvec(matrix, vector, result);
    print_vector(result);
    return 0;
}
```

OUTPUT:



```
PS C:\Users\arivu\OneDrive\Documents\MAP Lab> ./2b.exe
Arivuselvam S
710722104008

Parallel Implementation with OpenMP:
14.00 32.00 50.00
```

RESULT:

Thus the program that computes a parallelizing implementation of matrix - vector multiplication $b=Ax$, using OpenMP is executed and output verified successfully.

Ex No : 02 C**IMPLEMENTATION OF MATRIX-VECTOR****DATE :****MULTIPLICATION USING OPENMP IN RACE CONDITION****AIM:**

To create a program that computes the implementation of matrix-vector multiplication $b=Ax$, using OpenMP in Race Condition.

ALGORITHM:

Step 1: Initialize matrix A and vector V.

Step 2: Use OpenMP to parallelize computation.

Step 3: No protection for shared memory leads to inconsistent results.

Step 4: Store the results in R.

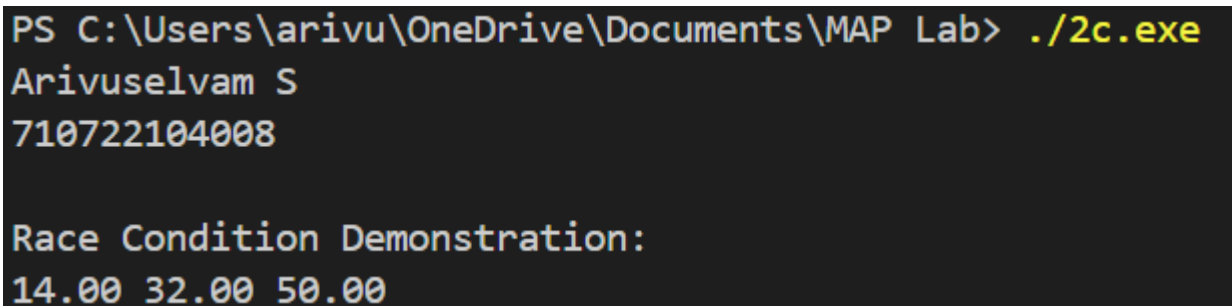
Step 5: Print the result.

PROGRAM:

```
#include <stdio.h>
#include <omp.h>
#define N 3 // Matrix Size
void race_condition_matvec(double matrix[N][N], double vector[N], double result[N]) {
    double temp[N] = {0}; // Shared resource without protection
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            temp[i] += matrix[i][j] * vector[j]; // Race condition here!
        }
    }
    // Copy result (incorrect due to race condition)
    for (int i = 0; i < N; i++) {
        result[i] = temp[i];
    }
}
void print_vector(double result[N]) {
```

```
        for (int i = 0; i < N; i++) {
            printf("%.2f ", result[i]);
        }
        printf("\n");
    }
int main() {
    double matrix[N][N] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };
    double vector[N] = {1, 2, 3};
    double result[N];
    printf("Race Condition Demonstration:\n");
    race_condition_matvec(matrix, vector, result);
    print_vector(result);
    return 0;
}
```

OUTPUT:



```
PS C:\Users\arivu\OneDrive\Documents\MAP Lab> ./2c.exe
Arivuselvam S
710722104008

Race Condition Demonstration:
14.00 32.00 50.00
```

RESULT:

Thus the program that computes a parallelizing implementation of matrix-vector multiplication $b = Ax$, using OpenMP in race condition is executed and output verified successfully.

Ex No : 03 A**DATE :****FLOYD'S ALGORITHM****AIM:**

To Implement the All-Pairs Shortest-Path Problem (Floyd's Algorithm) Using OpenMP.

ALGORITHM:

Step 1: Initialize the graph matrix with initial distances between vertices.

Step 2: Create a distance matrix dist of size N x N to store the shortest path distances.

Step 3: Copy the values from the graph matrix to the distance matrix.

Step 4: Parallelize the following loop using OpenMP directives: a. For each intermediate vertex k from 0 to N-1:

- a. Use #pragma omp parallel for directive to parallelize the outer loop.
- b. For each pair of vertices i and j from 0 to N-1:
- c. If dist[i][k] and dist[k][j] are not infinity:
- d. If the distance through vertex k is shorter than the current distance between i and j, update dist[i][j] to the new shorter distance.

Step 1: Print the resulting shortest path distances stored in the dist matrix.

PROGRAM:

```
#include <stdio.h>
#include <limits.h>
#include <omp.h>
#define INF 99999
#define N 4 // Number of vertices
void floyd_warshall_parallel(int graph[N][N]) {
    int dist[N][N], i, j, k;
    // Initialize distance matrix
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            dist[i][j] = graph[i][j];
    // Parallel Floyd-Warshall
```

```
for (k = 0; k < N; k++) {
    #pragma omp parallel for private(i, j) // OpenMP parallelization
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            if (dist[i][k] != INF && dist[k][j] != INF && dist[i][j] > dist[i][k] + dist[k][j]) {
                dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }
}

// Print shortest distance matrix
printf("Parallel Shortest Distance Matrix:\n");
for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++) {
        if (dist[i][j] == INF)
            printf("%4s", "INF");
        else
            printf("%4d", dist[i][j]);
    }
    printf("\n");
}

int main() {
    int graph[N][N] = {
        {0, 3, INF, 7},
        {8, 0, 2, INF},
        {5, INF, 0, 1},
        {2, INF, INF, 0}
    };
    floyd_warshall_parallel(graph);
    return 0;
}
```


OUTPUT:

```
PS C:\Users\arivu\OneDrive\Documents\MAP Lab> ./3a.exe
Arivuselvam S
710722104008

Parallel Shortest Distance Matrix:
  0  3  5  6
  5  0  2  3
  3  6  0  1
  2  5  7  0
```

RESULT:

Thus the implementation of All-Pairs Shortest-Path Problem (Floyd's Algorithm) Using OpenMP were executed successfully.

Ex No : 03 B	WORK SHARING IN PARALLEL FLOYD-WARSHALL ALGORITHM
DATE :	

Aim:

To implement the Floyd-Warshall algorithm using OpenMP Work Sharing to efficiently distribute computations among multiple threads.

ALGORITHM:

Step 1: Initialize the distance matrix `dist[][]` with given edge weights.

Step 2: Use OpenMP Work Sharing (`#pragma omp parallel for`) to distribute the workload across threads:

- Parallelize the inner loops (i and j) while keeping k sequential.

Step 3: Each thread computes a portion of the distance matrix independently.

Step 4: After N iterations, the matrix contains the shortest distances.

Step 5: Output the final shortest path matrix.

PROGRAM:

```
#include <stdio.h>
#include <limits.h>
#include <omp.h>
#define INF 99999
#define N 4 // Number of vertices
void floyd_warshall_work_sharing(int graph[N][N]) {
    int dist[N][N], i, j, k;
    // Initialize distance matrix
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            dist[i][j] = graph[i][j];
    // Work-Sharing Parallelization
    for (k = 0; k < N; k++) {
        #pragma omp parallel for shared(dist, k) private(i, j) schedule(dynamic)
        for (i = 0; i < N; i++) {
```

```
        for (j = 0; j < N; j++) {
            if (dist[i][k] != INF && dist[k][j] != INF && dist[i][j] > dist[i][k] + dist[k][j]) {
                dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }
}

// Print shortest distance matrix
printf("Work-Sharing Shortest Distance Matrix:\n");
for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++) {
        if (dist[i][j] == INF)
            printf("%4s", "INF");
        else
            printf("%4d", dist[i][j]);
    }
    printf("\n");
}

int main() {
    int graph[N][N] = {
        {0, 3, INF, 7},
        {8, 0, 2, INF},
        {5, INF, 0, 1},
        {2, INF, INF, 0}
    };

    floyd_warshall_work_sharing(graph);
    return 0;
}
```

OUTPUT:

```
PS C:\Users\arivu\OneDrive\Documents\MAP Lab> ./3b.exe  
Arivuselvam S  
710722104008
```

Work-Sharing Shortest Distance Matrix:

0	3	5	6
5	0	2	3
3	6	0	1
2	5	7	0

RESULT:

Thus the implementation of All-Pairs Shortest-Path Problem (Floyd's Algorithm) Using OpenMP Work Sharing were executed successfully.

Ex No : 03 C**HANDLING DEPENDENCIES IN PARALLEL FLOYD-****DATE :****WARSHALL ALGORITHM****AIM:**

To implement the Floyd-Warshall algorithm in parallel using OpenMP, while correctly handling dependencies between loop iterations.

ALGORITHM:

Step1: Initialize the distance matrix `dist[][]` from the input graph.

Step2: Iterate over each intermediate node `k` (sequentially) because updates in one iteration depend on the results of the previous iteration.

Step3: Use OpenMP to parallelize the inner loops (`i` and `j`) while ensuring that `dist[i][k]` and `dist[k][j]` are accessed consistently.

Step4: Update `dist[i][j]` using the Floyd-Warshall formula:

$$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$$

Step5: Print the shortest distance matrix.

PROGRAM:

```
#include <stdio.h>
#include <limits.h>
#include <omp.h>
#define INF 99999
#define N 4 // Number of vertices
void floyd_warshall_handling_dependencies(int graph[N][N]) {
    int dist[N][N], i, j, k;
    // Initialize distance matrix
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            dist[i][j] = graph[i][j];
    // Floyd-Warshall with dependency handling
```

```
for (k = 0; k < N; k++) {
    #pragma omp parallel for shared(dist, k) private(i, j)
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            if (dist[i][k] != INF && dist[k][j] != INF && dist[i][j] > dist[i][k] + dist[k][j]) {
                dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }
}

// Print shortest distance matrix
printf("Dependency-Handled Shortest Distance Matrix:\n");
for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++) {
        if (dist[i][j] == INF)
            printf("%4s", "INF");
        else
            printf("%4d", dist[i][j]);
    }
    printf("\n");
}

int main() {
    int graph[N][N] = {
        {0, 3, INF, 7},
        {8, 0, 2, INF},
        {5, INF, 0, 1},
        {2, INF, INF, 0}
    };
    floyd_warshall_handling_dependencies(graph);
    return 0;
}
```

OUTPUT:

```
PS C:\Users\arivu\OneDrive\Documents\MAP Lab> ./3c.exe  
Arivuselvam S  
710722104008
```

Dependency-Handled Shortest Distance Matrix:

0	3	5	6
5	0	2	3
3	6	0	1
2	5	7	0

RESULT:

Thus, the implement the Floyd-Warshall algorithm in parallel using OpenMP, while correctly handling dependencies between loop iterations.

Ex No : 04 A	PARALLEL RANDOM NUMBER GENERATORS SEQUENTIAL SIMULATION
DATE :	

AIM:

To estimate the value of π using the Sequential Monte Carlo method by generating random points and calculating the ratio of points that fall inside a unit circle.

ALGORITHM:

Step1: Define NUM_POINTS as the total number of random points to be generated.

Step2: Initialize a counter count = 0 to store the number of points that fall inside the unit circle.

Step3: Seed the random number generator using the current time (srand(time(NULL))).

Step4: Repeat the following steps for each point (from i = 0 to NUM_POINTS - 1):

- Generate a random x-coordinate between 0 and 1.
- Generate a random y-coordinate between 0 and 1.
- Check if the point (x, y) lies inside the unit circle using the condition:
$$x*x + y*y \leq 1$$
- If true, increment the counter count.

Step5: Calculate the estimated value of π using the formula:

$$\Pi \approx 4 \times \text{count} / \text{NUM_POINTS}$$

Step6: Display the estimated value of π .

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define NUM_POINTS 1000000 // You can increase for better accuracy

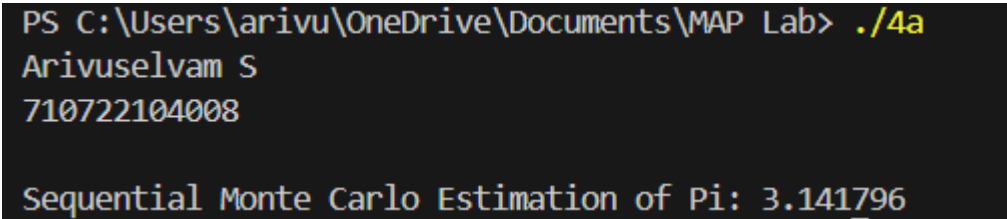
int main() {
    int i, count = 0;
    double x, y;
    srand(time(NULL));

    for (i = 0; i < NUM_POINTS; i++) {
        x = (double)rand() / RAND_MAX;
```



```
y = (double)rand() / RAND_MAX;
if (x * x + y * y <= 1.0) {
    count++;
}
}

double pi = 4.0 * count / NUM_POINTS;
printf("Arivuselvam S\n710722104008\n\n");
printf("Sequential Monte Carlo Estimation of Pi: %f\n", pi);
return 0;
}
```

OUTPUT:

```
PS C:\Users\arivu\OneDrive\Documents\MAP Lab> ./4a
Arivuselvam S
710722104008

Sequential Monte Carlo Estimation of Pi: 3.141796
```

RESULT:

Thus, the Sequential Monte Carlo simulation was successfully implemented to estimate the value of π by generating random points and calculating the ratio of points that fall inside a unit circle.

Ex No : 04 B	PARALLEL RANDOM NUMBER GENERATION USING OPENMP
DATE :	

AIM:

To estimate the value of π using the Monte Carlo method implemented in parallel using OpenMP by generating random points and computing the ratio of points that fall inside a unit circle.

ALGORITHM:

Step1: Define NUM_POINTS as the total number of random points to be generated.

Step2: Initialize a shared variable count = 0 to store the total number of points that fall inside the unit circle.

Step3: Begin a parallel region using #pragma omp parallel.

Step4: For each thread:

- Create a unique seed using `time(NULL) ^ omp_get_thread_num()` for thread-safe random number generation with `rand_r()`.
- Initialize a thread-local variable `local_count = 0`.

Step5: Use #pragma omp for to distribute iterations of the loop:

- Generate random x and y values between 0 and 1.
- Check if the point (x, y) lies within the unit circle using the condition:
$$x*x + y*y \leq 1$$
- If true, increment `local_count`.

Step6: Use #pragma omp atomic to safely add each thread's `local_count` to the global count.

Step7: After the parallel region, calculate π using the formula:

$$\Pi \approx 4 \times \text{count} / \text{NUM_POINTS}$$

Step8: Print the estimated value of π .

PROGRAM:**Program:**

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>
```

```
#define NUM_POINTS 100 // Total number of random points

int main() {
    int count = 0;
    srand(time(NULL)); // Seed the RNG (only once)
    #pragma omp parallel
    {
        int local_count = 0;
        double x, y;
        #pragma omp for
        for (int i = 0; i < NUM_POINTS; i++) {
            #pragma omp critical // Ensure rand() is thread-safe
            {
                x = (double)rand() / RAND_MAX;
                y = (double)rand() / RAND_MAX;
            }
            if (x * x + y * y <= 1.0) {
                local_count++;
            }
        }
        #pragma omp atomic
        count += local_count;
    }
    double pi = 4.0 * count / NUM_POINTS;
    printf("Arivuselvam S\n710722104008\n\n");
    printf("Parallel Monte Carlo Estimation of Pi (OpenMP): %f\n", pi);
    return 0;
}
```

OUTPUT:

```
PS C:\Users\arivu\OneDrive\Documents\MAP Lab> ./4b
Arivuselvam S
710722104008

Parallel Monte Carlo Estimation of Pi (OpenMP): 3.280000
```

RESULT:

Thus, the Monte Carlo simulation was successfully implemented using OpenMP to estimate the value of π in parallel.

Ex No : 05 A	MPI BROADCAST COMMUNICATION AND MPI REDUCE OPERATION
DATE :	

AIM:

To demonstrate the use of MPI_Bcast (broadcast communication) and MPI_Reduce (collective reduction operation) in a parallel program using the Message Passing Interface (MPI).

ALGORITHM:

Step1: Start the Program

Step2: MPI Initialization:

- Start MPI with MPI_Init.
- Get rank and size using MPI_Comm_rank and MPI_Comm_size.

Step3: Broadcast Communication:

- The root process (usually rank 0) sets a value.
- Use MPI_Bcast() to broadcast the value to all other processes.

Step4: Reduce Operation:

- Each process computes a partial value (e.g., its rank * broadcasted value).
- Use MPI_Reduce() to perform a sum of all these values and send the result to the root process.

Step5: Finalization:

- Root prints the result.
- Finalize using MPI_Finalize.

Step6: End the Program

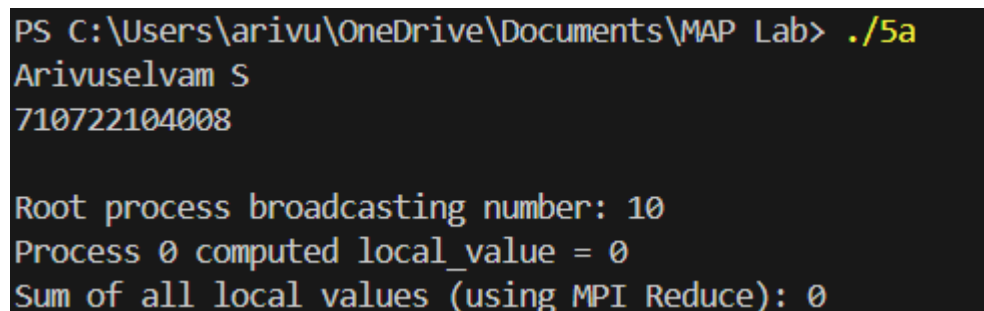
PROGRAM:

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char* argv[]) {
    printf("Arivuselvam S\n710722104008\n\n");
    int rank, size, root = 0;
    int number;
```

```
MPI_Init(&argc, &argv); // Initialize MPI environment
MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Get current process rank
MPI_Comm_size(MPI_COMM_WORLD, &size); // Get total number of processes
if (rank == root) {
    number = 10; // Root sets the value to broadcast
    printf("Root process broadcasting number: %d\n", number);
}
// Broadcast 'number' from root to all processes
MPI_Bcast(&number, 1, MPI_INT, root, MPI_COMM_WORLD);
// Each process does some computation (e.g., multiply by rank)
int local_value = rank * number;
printf("Process %d computed local_value = %d\n", rank, local_value);
int sum;
// Reduce local_value to root using sum operation
MPI_Reduce(&local_value, &sum, 1, MPI_INT, MPI_SUM, root, MPI_COMM_WORLD);
if (rank == root) {
    printf("Sum of all local values (using MPI_Reduce): %d\n", sum);
}
MPI_Finalize(); // Clean up the MPI environment
return 0;
}
```

OUTPUT:



```
PS C:\Users\arivu\OneDrive\Documents\MAP Lab> ./5a
Arivuselvam S
710722104008

Root process broadcasting number: 10
Process 0 computed local_value = 0
Sum of all local values (using MPI_Reduce): 0
```

RESULT:

Thus, the simple Program to demonstrate the use of MPI_Bcast (broadcast communication) and MPI_Reduce (collective reduction operation) in a parallel program using the Message Passing Interface (MPI) was executed and output verified successfully.

Ex No : 05 B	MPI SCATTER, GATHER COMMUNICATION, AND COMBINATION OF OPERATIONS IN C
DATE :	

AIM:

To demonstrate MPI Scatter and MPI Gather operations with local computation, and to show how a combination of collective communication and reduction operations can be used for performance in parallel programs.

ALGORITHM:

Step1: Start the Program

Step2: Initialize MPI and get rank/size.

Step3: Root process prepares an array of N elements (where N is divisible by the number of processes).

Step4: Use MPI_Scatter to send chunks of data to each process.

Step5: Each process:

- Computes a local sum of its data.

Step6: Use MPI_Gather to collect local sums to root.

Step7: Use MPI_Reduce to get global sum in root process.

Step8: Root prints gathered values and total sum.

Step9: Finalize MPI

Step10: End the Program

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <mpi.h>
```

```
#define N 8 // Total number of elements (must be divisible by number of processes)
```

```
int main(int argc, char* argv[]) {
```

```
    int rank, size;
```

```
    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
int chunk_size = N / size;
int data[N];
int recv_buf[chunk_size];
printf("Arivuselvam S\n710722104008\n\n");

if (rank == 0) {
    // Initialize data only in root
    printf("Original data array:\n");
    for (int i = 0; i < N; i++) {
        data[i] = i + 1;
        printf("%d ", data[i]);
    }
    printf("\n");
}

// Scatter chunks to all processes
MPI_Scatter(data, chunk_size, MPI_INT, recv_buf, chunk_size, MPI_INT, 0,
MPI_COMM_WORLD);

// Each process computes sum of its chunk
int local_sum = 0;
for (int i = 0; i < chunk_size; i++) {
    local_sum += recv_buf[i];
}

printf("Process %d local sum = %d\n", rank, local_sum);

// Gather local sums back to root
int gathered_sums[size];
MPI_Gather(&local_sum, 1, MPI_INT, gathered_sums, 1, MPI_INT, 0,
MPI_COMM_WORLD);

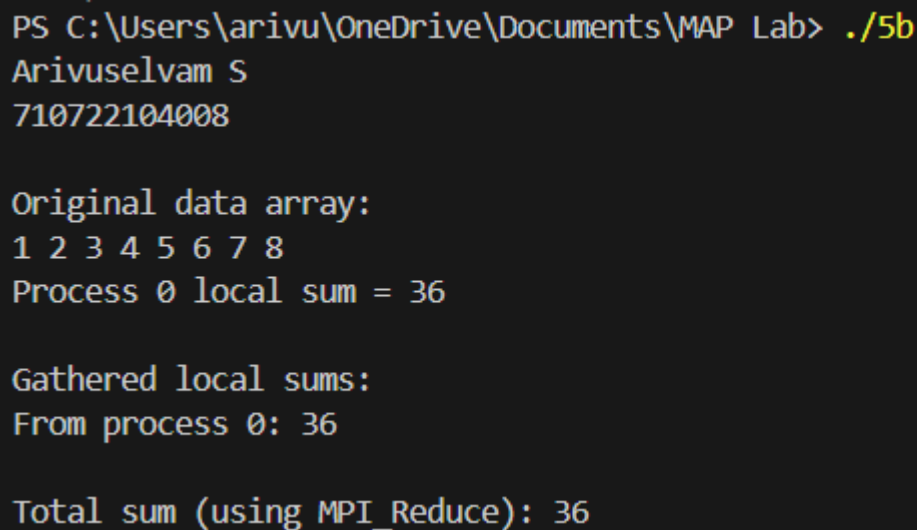
// Also compute global sum using reduce
int global_sum;
MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0,
MPI_COMM_WORLD);

// Root prints all local sums and total sum
if (rank == 0) {
    printf("\nGathered local sums:\n");
    for (int i = 0; i < size; i++) {
```



```
        printf("From process %d: %d\n", i, gathered_sums[i]);
    }
    printf("\nTotal sum (using MPI_Reduce): %d\n", global_sum);
}
MPI_Finalize();
return 0;
}
```

OUTPUT:



```
PS C:\Users\arivu\OneDrive\Documents\MAP Lab> ./5b
Arivuselvam S
710722104008

Original data array:
1 2 3 4 5 6 7 8
Process 0 local sum = 36

Gathered local sums:
From process 0: 36

Total sum (using MPI_Reduce): 36
```

RESULT:

Thus the program to demonstrate MPI Scatter and MPI Gather operations with local computation, and to show how a combination of collective communication and reduction operations can be used for performance in parallel programs was executed and output verified successfully.

Ex No : 06 A**DATE :****BASIC MPI SETUP****AIM:**

To implement a basic MPI setup to initialize and finalize the MPI environment and print the rank and size of each process

ALGORITHM:

Step 1: Start the Program

Step 2: Initialize the MPI environment using MPI_Init.

Step 3: Get the rank of the process using MPI_Comm_rank.

Step 4: Get the total number of processes using MPI_Comm_size.

Step 5: Print the process rank and total number of processes.

Step 6: Finalize the MPI environment using MPI_Finalize.

Step 7: Stop the Program.

PROGRAM:

```
#include <stdio.h>
```

```
#include <mpi.h>
```

```
int main(int argc, char *argv[]) {
```

```
    printf("Arivuselvam S\n710722104008\n\n");
```

```
    int rank, size;
```

```
    // Step 1: Initialize MPI environment
```

```
    MPI_Init(&argc, &argv);
```

```
    // Step 2: Get the number of processes
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
    // Step 3: Get the rank of the current process
```

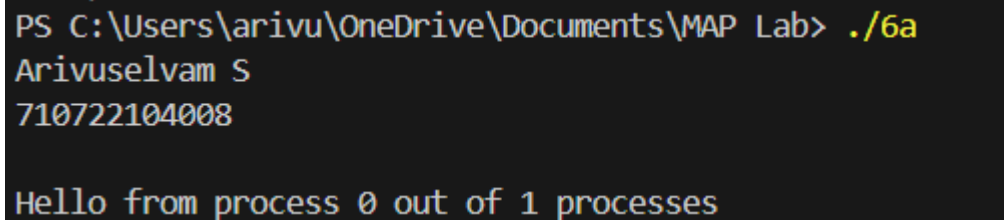
```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    // Step 4: Print message from each process
```

```
    printf("Hello from process %d out of %d processes\n", rank, size);
```

```
    // Step 5: Finalize MPI environment
```

```
MPI_Finalize();  
return 0;  
}
```

OUTPUT:A screenshot of a Windows command prompt window. The title bar is not visible. The command prompt shows the current directory as 'C:\Users\arivu\OneDrive\Documents\MAP Lab'. The user has entered the command './6a' and pressed enter. The output of the program is displayed on three lines: 'Arivuselvam S', '710722104008', and 'Hello from process 0 out of 1 processes'.

```
PS C:\Users\arivu\OneDrive\Documents\MAP Lab> ./6a  
Arivuselvam S  
710722104008  
Hello from process 0 out of 1 processes
```

RESULT:

Thus the program to implement a basic MPI setup to initialize and finalize the MPI environment and print the rank and size of each process is executed and output verified successfully.

Ex No : 06 B**DATE :****MPI SCATTER****AIM:**

To demonstrate how to distribute data from a root process to all other processes using MPI_Scatter.

ALGORITHM:

Step 1: Initialize the MPI environment.

Step 2: Root process prepares an array of data.

Step 3: Use MPI_Scatter to distribute chunks of the array to all processes.

Step 4: Each process prints the data it received.

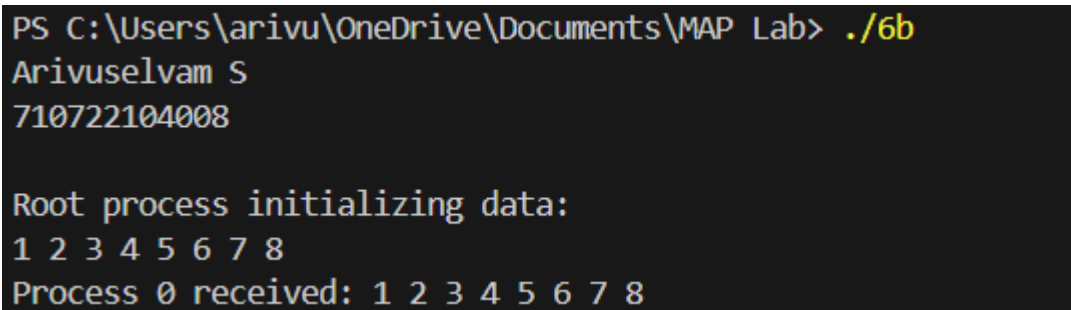
Step 5: Finalize MPI.

PROGRAM:

```
#include <stdio.h>
#include <mpi.h>
#define N 8 // Total number of elements (must be divisible by number of processes)
int main(int argc, char *argv[]) {
    printf("Arivuselvam S\n710722104008\n\n");
    int rank, size;
    int data[N];
    int recv_buf[N]; // assuming max N elements
    MPI_Init(&argc, &argv); // Initialize MPI
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Get current process rank
    MPI_Comm_size(MPI_COMM_WORLD, &size); // Get total number of processes
    int chunk_size = N / size;
    // Root process initializes data
    if (rank == 0) {
        printf("Root process initializing data:\n");
        for (int i = 0; i < N; i++) {
            data[i] = i + 1;
        }
    }
}
```

```
        printf("%d ", data[i]);
    }
    printf("\n");
}
// Scatter data to all processes
MPI_Scatter(data, chunk_size, MPI_INT, recv_buf, chunk_size, MPI_INT, 0,
MPI_COMM_WORLD);
// Each process prints its received data
printf("Process %d received: ", rank);
for (int i = 0; i < chunk_size; i++) {
    printf("%d ", recv_buf[i]);
}
printf("\n");
MPI_Finalize(); // Finalize MPI
return 0;
}
```

OUTPUT:



```
PS C:\Users\arivu\OneDrive\Documents\MAP Lab> ./6b
Arivuselvam S
710722104008

Root process initializing data:
1 2 3 4 5 6 7 8
Process 0 received: 1 2 3 4 5 6 7 8
```

RESULT:

Thus the program to implement a basic MPI setup to initialize and finalize the MPI environment and print the rank and size of each process is executed and output verified successfully.

Ex No : 06 C**DATE :****MPI GATHER AND ALLGATHER****AIM:**

To implement MPI collective communication operations where MPI_Gather collects data from all processes into the root process, and MPI_Allgather collects and distributes data from all processes to every process in a parallel MPI environment.

ALGORITHM:

Step 1: Initialize the MPI environment.

Step 2: Each process creates its own data (e.g., rank + 1).

Step 3: Use MPI_Gather to collect one value from each process at the root.

Step 4: Use MPI_Allgather to collect one value from each process and share it with all processes.

Step 5: Print the results

Step 6: Finalize MPI.

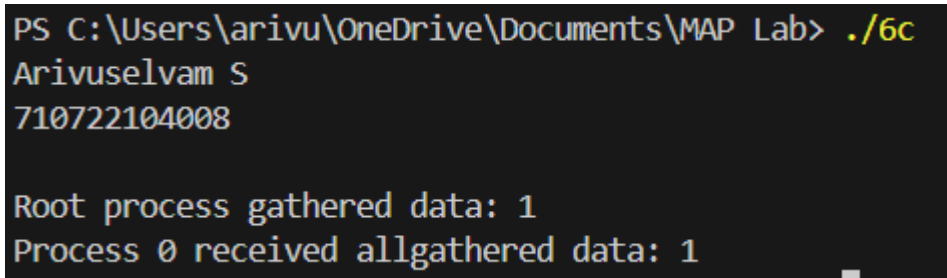
PROGRAM:

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    printf("Arivuselvam S\n710722104008\n\n");
    int rank, size;
    int send_data, gathered_data[100], allgathered_data[100]; // assume max 100 processes
    MPI_Init(&argc, &argv); // Initialize MPI
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Get rank
    MPI_Comm_size(MPI_COMM_WORLD, &size); // Get total number of processes
    send_data = rank + 1; // Each process sends its rank + 1
    // Gather at root
    MPI_Gather(&send_data, 1, MPI_INT, gathered_data, 1, MPI_INT, 0,
    MPI_COMM_WORLD);
    if (rank == 0) {
        printf("Root process gathered data: ");
```

```
        for (int i = 0; i < size; i++) {
            printf("%d ", gathered_data[i]);
        }
        printf("\n");
    }
    // Allgather to all processes
    MPI_Allgather(&send_data, 1, MPI_INT, allgathered_data, 1, MPI_INT,
    MPI_COMM_WORLD);
    // Each process prints the allgathered data
    printf("Process %d received allgathered data: ", rank);
    for (int i = 0; i < size; i++) {
        printf("%d ", allgathered_data[i]);
    }
    printf("\n");
    MPI_Finalize(); // Finalize MPI
    return 0;
}
```

OUTPUT:



```
PS C:\Users\arivu\OneDrive\Documents\MAP Lab> ./6c
Arivuselvam S
710722104008

Root process gathered data: 1
Process 0 received allgathered data: 1
```

RESULT:

Thus the program to implement MPI collective communication operations where MPI_Gather collects data from all processes into the root process, and MPI_Allgather collects and distributes data from all processes to every process in a parallel MPI environment. were executed successfully.